18.2 块流（Block Streaming）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~500,000 tokens，输出 ~40,000 tokens（本章合计）

模型生成的文本是一个连续的 token 流，但消息通道（如 Telegram、Signal）需要的是**离散的消息**。将连续文本流切分为合适大小的消息块——这就是 **块流（Block Streaming）** 要解决的核心问题。

OpenClaw 的 `EmbeddedBlockChunker` 实现了一套精密的文本分块算法，它不是简单地按固定字符数切分，而是会寻找**语义上合理的断点**，并且能够安全地处理 Markdown 代码围栏。

18.2.1 EmbeddedBlockChunker 算法（`src/agents/pi-embedded-block-chunker.ts`）

BlockReplyChunking 配置

```
// src/agents/pi-embedded-block-chunker.ts
export type BlockReplyChunking = {
  minChars: number;        // 最小块大小
  maxChars: number;        // 最大块大小
  breakPreference?: "paragraph" | "newline" | "sentence";
  flushOnParagraph?: boolean; // 遇到段落边界立即刷新
};
```

`minChars` 和 `maxChars` 构成了一个**水位线区间**：文本在 `minChars` 以下时继续积累，超过 `maxChars` 时强制断开，在两者之间时寻找合适的断点。

核心类结构

```
export class EmbeddedBlockChunker {
  #buffer = "";                    // 文本缓冲区
  readonly #chunking: BlockReplyChunking; // 配置

  append(text: string) {           // 追加文本
    this.#buffer += text;
  }

  reset() {                        // 重置缓冲区
    this.#buffer = "";
  }

  hasBuffered(): boolean {         // 是否有未发送的内容
    return this.#buffer.length > 0;
  }

  drain(params: {                  // 排出缓冲区
    force: boolean;                // 是否强制（忽略 minChars）
    emit: (chunk: string) => void; // 发射回调
  }) { ... }
}
```

drain 方法的执行流程

```
drain(params: { force: boolean; emit: (chunk: string) => void }) {
  const minChars = Math.max(1, Math.floor(this.#chunking.minChars));
  const maxChars = Math.max(minChars, Math.floor(this.#chunking.maxChars));

  // 段落模式：遇到 \n\n 立即刷新
  if (this.#chunking.flushOnParagraph && !force) {
    this.#drainParagraphs(emit, maxChars);
    return;
  }

  // 低于低水位线且非强制 → 继续积累
  if (this.#buffer.length < minChars && !force) return;

  // 强制且在限制内 → 整体发送
  if (force && this.#buffer.length <= maxChars) {
    if (this.#buffer.trim().length > 0) emit(this.#buffer);
    this.#buffer = "";
    return;
  }

  // 循环切分
  while (this.#buffer.length >= minChars ||
         (force && this.#buffer.length > 0)) {
    const breakResult = force && this.#buffer.length <= maxChars
      ? this.#pickSoftBreakIndex(this.#buffer, 1)
      : this.#pickBreakIndex(this.#buffer, force ? 1 : undefined);

    if (breakResult.index <= 0) {
      if (force) { emit(this.#buffer); this.#buffer = ""; }
      return;
    }

    this.#emitBreakResult(breakResult, emit);

    if (this.#buffer.length < minChars && !force) return;
  }
}
```

18.2.2 低水位线 / 高水位线分块策略

分块策略基于经典的**水位线**（Watermark）模式：

```
maxChars（高水位线）
──────────────────────────
↑ 必须断开（强制寻找断点）
│
│ 偏好区间（在此区间寻找最佳断点）
│
──────────────────────────
↑ minChars（低水位线）
│ 继续积累（不切分）
──────────────────────────
```

- **低于 minChars**：不切分，继续积累文本
- **minChars ~ maxChars 之间**：寻找**最佳断点**——从高到低依次尝试段落、换行、句子、空白
- **超过 maxChars**：强制在 maxChars 处切分，即使在代码块内部也要切（使用围栏重开机制）

这种设计的优势是：短消息不会被过早拆分（避免碎片化），长消息也不会无限积累（避免超时或通道限制）。

> **衍生解释——水位线（Watermark）模式**
>
> 水位线模式源自流量控制领域。低水位线表示"开始行动"的阈值，高水位线表示"必须行动"的阈值。在网络编程中，TCP 发送缓冲区的低/高水位线控制着何时通知应用可以写入数据。OpenClaw 借用这一概念来控制文本块的大小。

18.2.3 断点偏好：段落 → 换行 → 句子 → 空白 → 硬断

`#pickBreakIndex` 方法按优先级依次尝试不同类型的断点：

```
#pickBreakIndex(buffer: string, minCharsOverride?: number): BreakResult {
  const window = buffer.slice(0, Math.min(maxChars, buffer.length));
  const fenceSpans = parseFenceSpans(buffer);
  const preference = this.#chunking.breakPreference ?? "paragraph";

  // 优先级 1: 段落断点 (\n\n)
  if (preference === "paragraph") {
    let paragraphIdx = window.lastIndexOf("\n\n");
    while (paragraphIdx >= minChars) {
      if (isSafeFenceBreak(fenceSpans, paragraphIdx)) {
        return { index: paragraphIdx };
      }
      paragraphIdx = window.lastIndexOf("\n\n", paragraphIdx - 1);
    }
  }

  // 优先级 2: 换行断点 (\n)
  if (preference === "paragraph" || preference === "newline") {
    let newlineIdx = window.lastIndexOf("\n");
    while (newlineIdx >= minChars) {
      if (isSafeFenceBreak(fenceSpans, newlineIdx)) {
        return { index: newlineIdx };
      }
      newlineIdx = window.lastIndexOf("\n", newlineIdx - 1);
    }
  }

  // 优先级 3: 句子断点 (.!? 后跟空白或行尾)
  if (preference !== "newline") {
    const matches = window.matchAll(/[.!?](?=\s|$)/g);
    // 选择最后一个有效句子断点
    let sentenceIdx = -1;
    for (const match of matches) {
      const candidate = (match.index ?? -1) + 1;
      if (candidate >= minChars && isSafeFenceBreak(fenceSpans, candidate)) {
        sentenceIdx = candidate;
      }
    }
    if (sentenceIdx >= minChars) return { index: sentenceIdx };
  }

  // 优先级 4: 空白断点
  for (let i = window.length - 1; i >= minChars; i--) {
```

```
    if (/\s/.test(window[i]) && isSafeFenceBreak(fenceSpans, i)) {
      return { index: i };
    }
  }

  // 优先级 5：硬断（maxChars 处强制切分）
  if (buffer.length >= maxChars) {
    if (isSafeFenceBreak(fenceSpans, maxChars)) {
      return { index: maxChars };
    }
    // 在围栏内 → 使用围栏拆分
    const fence = findFenceSpanAt(fenceSpans, maxChars);
    if (fence) {
      return {
        index: maxChars,
        fenceSplit: {
          closeFenceLine: `${fence.indent}${fence.marker}`,
          reopenFenceLine: fence.openLine,
        },
      };
    }
    return { index: maxChars };
  }

  return { index: -1 }; // 无法断开
}
```

每个断点候选都要先过 `isSafeFenceBreak` 这一关——确保断点不在 Markdown 代码围栏内部。在代码块中间断开会导致 Markdown 渲染异常，所以这个检查不能省。

段落急切刷新模式

当 `flushOnParagraph` 为 `true` 时，每遇到一个段落边界（`\n\n`）就立即发送，不等待缓冲区填满：

```
#drainParagraphs(emit: (chunk: string) => void, maxChars: number) {
  while (this.#buffer.length > 0) {
    const fenceSpans = parseFenceSpans(this.#buffer);
    const paragraphBreak = findNextParagraphBreak(this.#buffer, fenceSpans);

    if (!paragraphBreak || paragraphBreak.index > maxChars) {
      // 没有段落边界或太远 → 回退到普通断点逻辑
      if (this.#buffer.length >= maxChars) {
        const breakResult = this.#pickBreakIndex(this.#buffer, 1);
        if (breakResult.index > 0) {
          this.#emitBreakResult(breakResult, emit);
          continue;
        }
      }
      return;
    }

    const chunk = this.#buffer.slice(0, paragraphBreak.index);
    if (chunk.trim().length > 0) emit(chunk);
    this.#buffer = stripLeadingNewlines(
      this.#buffer.slice(paragraphBreak.index + paragraphBreak.length)
    );
  }
}
```

18.2.4 代码围栏（Fenced Block）安全拆分：关闭 + 重开

代码围栏安全拆分是块流算法里最有意思的部分。当文本超过 maxChars 但断点恰好落在代码围栏内时，不能简单地在中间切断——这会破坏 Markdown 格式。解决方案是**关闭当前围栏，然后在下一块重新打开**：

```
#emitBreakResult(breakResult: BreakResult, emit) {
  let rawChunk = this.#buffer.slice(0, breakResult.index);
  let nextBuffer = this.#buffer.slice(breakResult.index);

  const fenceSplit = breakResult.fenceSplit;
  if (fenceSplit) {
    // 在当前块末尾添加关闭围栏
    const closeFence = rawChunk.endsWith("\n")
      ? `${fenceSplit.closeFenceLine}\n`
      : `\n${fenceSplit.closeFenceLine}\n`;
    rawChunk = `${rawChunk}${closeFence}`;

    // 在下一块开头添加重开围栏
    const reopenFence = fenceSplit.reopenFenceLine.endsWith("\n")
      ? fenceSplit.reopenFenceLine
      : `${fenceSplit.reopenFenceLine}\n`;
    nextBuffer = `${reopenFence}${nextBuffer}`;
  }

  emit(rawChunk);
  this.#buffer = fenceSplit ? nextBuffer : stripLeadingNewlines(...);
}
```

举例说明。假设原始文本为：

```
这是一些解释文字。

```python
def hello():
    print("Hello")
    # 这是一个很长的函数...
    # ...很多行代码...
    return True
```
```

如果 maxChars 恰好在 `print("Hello")` 后面，分块器会生成：

块 1：

这是一些解释文字。

```python
def hello():
    print("Hello")
```

块 2：

```python
    # 这是一个很长的函数...
    # ...很多行代码...
    return True
```

围栏拆分信息来自 `parseFenceSpans` 和 `findFenceSpanAt` ，它们解析 Markdown 中所有的围栏代码块并记录其位置、缩进和标记符：

```
type FenceSplit = {
  closeFenceLine: string;  // 关闭行（如 "```"）
  reopenFenceLine: string; // 重开行（如 "```python"）
};
```

本节小结

1. **EmbeddedBlockChunker** 将连续的文本流切分为适合消息通道发送的离散块。
2. **水位线策略**（minChars/maxChars）避免了消息过碎或过大的问题。
3. **断点优先级**（段落 > 换行 > 句子 > 空白 > 硬断）确保文本在语义合理的位置断开。
4. **围栏安全检查**（ `isSafeFenceBreak` ）防止在代码块内部断开，避免 Markdown 渲染异常。
5. **围栏关闭 + 重开**机制是块流算法的精华——在不得不切分代码块时，通过自动关闭和重开围栏保持 Markdown 的有效性。